

A3-PS034-TC501

Schema-Validated YANG-Push Telemetry Pipeline

Trishit Ghosh
`trishit.org@gmail.com`
Sinjini Datta
`sinjini.archive@gmail.com`

AIORI Hackathon 3
github.com/aiori-hackathon/A3-PS034-TC501
October 13, 2026

Problem Description

Clear outline of the problem being addressed.

- SNMP polling samples devices at coarse intervals and **misses transient state changes**; polling cycles get lost exactly when the network is under stress (RFC 8641 §1).
- YANG-Push streams **event-driven** updates, but high-speed telemetry can carry **malformed or schema-invalid** records into monitoring platforms.
- Brokers and consumers fail: restarts can cause **message loss, duplicates and added delay**, and these are rarely measured.
- Generic JSON is often mislabelled as "YANG-Push" – the input must be a real `push-update` / `push-change-update` notification.
- **Goal:** a containerised pipeline that validates, relays and measures YANG-Push telemetry end to end.

Background

YANG-Push (RFC 8641) replaces periodic polling with subscribed, pushed datastore updates.

- **Periodic** subscriptions push a full `push-update` every `period`; **on-change** ones push `push-change-update` (YANG Patch edits) with an optional `dampening-period` [1].
- Each notification carries the subscription `id`; on-change patches carry a `patch-id` counter that exposes lost or out-of-order records (§3.7).
- Publishers must flag gaps using `incomplete-update` and must not resequence updates for one subscription (§3.11.1).
- Payloads are governed by YANG models, so a YANG-aware validator can reject malformed data *before* it reaches a dashboard.
- Scope: **one** publisher, **one** YANG model, **one** broker, **one** dashboard – no multi-vendor or production-throughput claims.

Literature & RFC Review

Only the most relevant standards and prior work.

- **RFC 8641** [1] – YANG-Push: datastore subscriptions, `push-update`, `push-change-update`, reliability rules.
- **RFC 8639** [2] – Subscribed notifications framework underneath YANG-Push; transport-independent subscription model.
- **RFC 7950** [3] – YANG 1.1: the language used to define the model that payloads are validated against.
- **RFC 8072** [4] – YANG Patch: structure of on-change update records (`edit`, `operation`, `target`).
- **RFC 8040** [5] – RESTCONF; defines the JSON encoding of YANG data used by our emulated publisher.
- **IETF 126 Hackathon** – reference for YANG-Push interoperability and tooling practice.
- **Gap:** existing work focuses on publisher/receiver interoperability; delivery behaviour of validated telemetry across a broker during consumer restarts is not quantified.

Proposed Solution

Detailed description of the overall solution – four containers orchestrated by Docker Compose.

- **Publisher + Validator (custom Python module):** a faithful YANG-Push emulator emits RFC 8641 notifications (JSON encoding, `ietf-yang-push`); the same module validates every payload against the selected YANG model using a Python schema-validation library, and **rejects** malformed or schema-invalid records to a dead-letter topic.
- **Message broker – Apache Kafka:** accepted notifications are produced to a `yang-push.valid` topic, preserving the original notification structure (`id`, `eventTime`, `datastore-contents` / `yang-patch`).
- **Dashboard consumer – Next.js:** a Kafka consumer service exposes a live dashboard (accepted vs. rejected counts, loss, duplicates, delivery delay).
- **Containerisation – Docker:** every component has its own image; one `docker compose up` brings up the whole pipeline.
- **Measurement:** sequence IDs and timestamps give loss, duplicate and delay metrics, including a controlled **consumer-restart** test.

Optimization Proposed

List of optimizations and enhancements – before vs. after.

Aspect	Before (baseline)	After (proposed)
Data collection	SNMP polling, coarse intervals	YANG-Push streaming, event-driven
Input quality	Unchecked JSON reaches the dashboard	Schema-validated; invalid goes to dead-letter topic
Restart handling	Gaps / duplicates unnoticed	Committed offsets + sequence IDs; loss and duplicates measured
Delivery	Direct coupling to consumer	Kafka decouples and buffers
Deployment	Manual setup	Single `docker compose up`

Solution Architecture & Design

Visual: Flowchart or diagram showing the optimization process.

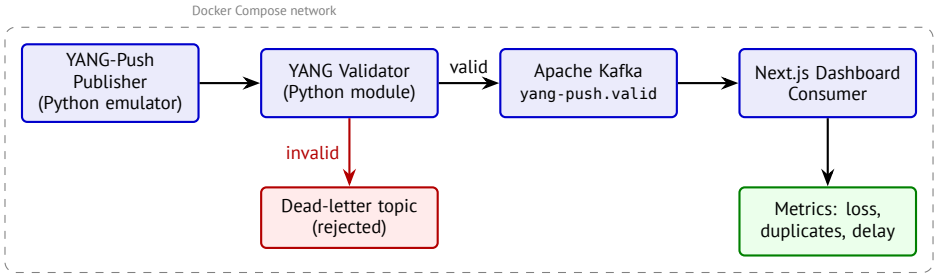


Figure: System Architecture UML Diagram

Data Flow Diagram

Overview of the architecture and the end-to-end flow of a notification.

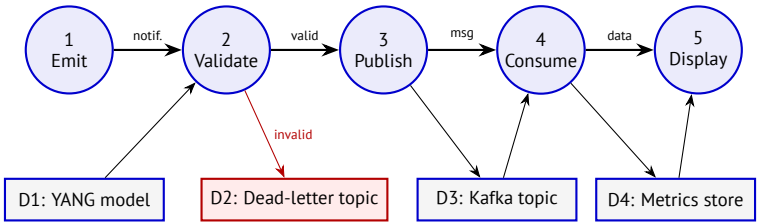


Figure: Data Flow Diagram (DFD)

Prototype Pseudocode Snippet

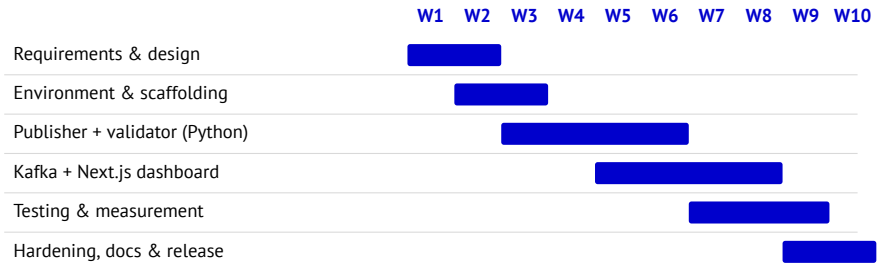
Core validate-and-relay loop (custom Python module).

```
for notif in publisher.stream():            # YANG-Push push-update / push-change-update
    if not is_yang_push(notif):             # reject generic JSON
        send(dlq, notif, reason="not-yang-push"); continue
    try:
        model.validate(notif.payload)       # ietf-yang-push + selected YANG model
        kafka.produce("yang-push.valid", key=notif.id, value=notif, headers={"seq": n, "sent_at": now()})
    except ValidationError as e:
        send(dlq, notif, reason=str(e))

# Next.js consumer: loss = missing seq, duplicate = repeated seq, delay = recv_ts - sent_at
```

Deployment Timeline

Project plan: 10 weeks, iterative one-week sprints, containerised and version-controlled from week 1.



W1–2	Requirements, YANG model selection, repo and Docker Compose skeleton, Kafka (KRaft) up
W3–6	Python YANG-Push emulator, schema-validation module, dead-letter topic, Kafka producer
W5–8	Kafka consumer service and Next.js dashboard (live counts, loss, duplicates, delay)
W7–9	Valid/malformed test cases, consumer-restart scenario, measurements, CI test runs
W9–10	Schema-validation and delivery report, documentation, tagged release, final demo

Bibliography I I

[1] A. Clemm and E. Voit. “Subscription to YANG Notifications for Datastore Updates.” RFC 8641, IETF, Sept. 2019.

[2] E. Voit et al. “Subscription to YANG Notifications.” RFC 8639, IETF, Sept. 2019.

[3] M. Bjorklund (Ed.). “The YANG 1.1 Data Modeling Language.” RFC 7950, IETF, Aug. 2016.

[4] A. Bierman, M. Bjorklund, K. Watsen. “YANG Patch Media Type.” RFC 8072, IETF, Feb. 2017.

[5] A. Bierman, M. Bjorklund, K. Watsen. “RESTCONF Protocol.” RFC 8040, IETF, Jan. 2017.

Thank You for Your Attention!